Accelerating the Krusell–Smith Method

Yoshiya Yokomoto*

October 2026

Abstract

This paper substantially accelerates the Krusell–Smith method, particularly the simulation stage in models with nontrivial market-clearing conditions. By precomputing agents' optimal responses to different prices, the method eliminates the need to repeatedly solve their decision problems during simulation. In the Bloom et al. (2018) model, the proposed grid-based method reduces simulation time from 290 minutes to 1.5 minutes and total computation time from approximately four days to 108 minutes. Because grid-based precomputation becomes costly as the number of states and equilibrium prices increases, I extend the approach using neural networks with a discrete–continuous policy representation that accommodates (S, s)-type adjustment rules. I then use the method to revisit aggregate comovement after a pure uncertainty shock in the Bloom et al. (2018) model. While the original specification fails to generate simultaneous declines in output, consumption, investment, and employment on impact, allowing for a finite Frisch elasticity restores this comovement.

Keywords. Krusell–Smith method; Heterogeneous-agent models; Neural networks; Aggregate uncertainty; Nonconvex adjustment costs

JEL Classification. C63, C68, E22, E32

*Keio University. Email: yoshiyayokomoto@gmail.com.

I am indebted to my adviser Tatsuro Senga for his invaluable guidance and support. I am also grateful for comments and suggestions from Marlon Azinovic, Ji Huang, Hidehiko Matsumoto, Jacob Røpke, Simon Scheidegger, Takashi Shiono, Yucheng Yang, Jan Žemlička, and participants at conferences and workshops. This research was supported by a grant from the Koizumi and Ushioda Memorial Fund at Keio University. All remaining errors are mine. An earlier version of this paper circulated under the title "Accelerating the Krusell–Smith Algorithm with Deep Learning."

1 Introduction

Heterogeneous firm models are widely used for studying the macroeconomic implications of micro-level heterogeneity among plants and firms; however, analyzing cyclical dynamics using this class of models with aggregate uncertainty poses a significant computational challenge. As established by Terry (2017), the Krusell-Smith (KS) method is the superior method among alternatives—particularly for handling large shocks and non-linear dynamics—offering unmatched accuracy and flexibility for adapting to diverse macroeconomic heterogeneous agent models. This approach can capture complex phenomena like occasionally binding constraints, non-stationary dynamics, and multiple equilibria that simpler linearization methods miss. Ironically, this same generality makes it the most computationally resource-demanding method required to achieve such precision.[1]

This paper makes three contributions. First, I develop a grid-based method that substantially accelerates the KS method while preserving its basic structure and benchmark accuracy. In the Bloom et al. (2018) model, total computation time falls from 6,141 minutes, or approximately four days, to 108 minutes. Second, I extend the method using neural networks, named Neural Network-based Krusell-Smith Method (NN-KS), to accommodate larger models. Additionally, I construct a policy representation that allows for the discontinuities generated by fixed adjustment costs. Third, I use this NN-KS method to extend household preferences in the model of Bloom et al. (2018) and show that a pure uncertainty shock can generate comovement in output, consumption, investment, and employment on impact, without an accompanying adverse first-moment shock.

Many economic models do not yield explicit formulae for agents' decisions, prices, and allocations. This difficulty is particularly acute for heterogeneous firm models because closed-form solutions for market-clearing prices cannot be derived. This is problematic for the Krusell-Smith algorithm because it relies on a long simulation of the stochastic economy wherein, in each period, we must find the market-clearing equilibrium prices (such as wages) numerically. This contrasts with standard incomplete market, household-centric heterogeneous agent models, where price determination is often simpler even under aggregate uncertainty.

In more detail, the standard approach requires solving the firm's decision problem given a guessed price in each simulation period. We then use the solved policy rules to obtain the distribution of firms and consequent aggregate quantities to check if market clearing is satisfied. Since the initial guess is rarely correct, we must iteratively repeat this process to find the market-clearing prices before moving to the next period. We then carry the resulting distribution of firms forward and restart the numerical search for the next period's prices, repeating this over the entire stochastic simulation of 5,000 to 10,000 periods. This process is incredibly demanding: under

[1] Terry (2017) systematically compares five solution methods for heterogeneous firm models—the traditional Krusell-Smith algorithm (Krusell and Smith (1998), Krusell and Smith (1997)), Parameterized Distributions (Algan et al., 2008), Explicit Aggregation (Den Haan and Rendahl, 2010), Projection plus Perturbation (Reiter, 2009), and Parameterization plus Perturbation (Winberry, 2018).

the standard Krusell–Smith method, simulating the model in Bloom et al. (2018) for 5,000 periods takes 290 minutes, compared with just 7 minutes to solve its Bellman equation.

The novelty of the proposed method is that it resolves this exact bottleneck by rendering the repetitive solution of the firm's decision problem unnecessary during the price search. Specifically, I include prices directly as arguments in the firm's policy functions alongside moments of distribution and precompute these policy functions before simulation. This allows the simulation loop to determine the market-clearing price quickly via the policy function, bypassing the time-consuming optimization problem required to derive decision rules at every guess. With the proposed method, the same simulation takes just 1.5 minutes.

However, precomputing policies on a grid becomes costly as the number of individual states, aggregate moments, and prices increases. The number of entries in the policy table grows multiplicatively with these dimensions. In practice, the Bloom et al. (2018) model is close to the largest model for which this precomputation is feasible, given its computational and memory requirements. I therefore approximate both the value function and the price-conditioned policy function with neural networks. Once trained, evaluating the policy for any candidate price becomes a simple forward pass through the network. A key contribution is a neural-network architecture that represents discontinuous policy rules, such as (S, s) rules. Specifically, I decompose the firm's decision into a discrete choice of whether to adjust and a continuous choice of how much to adjust, conditional on adjustment. The discrete policy network selects an adjustment regime, and the conditional policy network determines the action within that regime. This representation preserves exact inaction and allows the policy to jump at adjustment thresholds, which a single standard neural network cannot represent exactly. NN–KS completes the Bloom et al. (2018) application in 183 minutes. Its principal advantage over the accelerated grid method is the ability to accommodate additional state variables and equilibrium prices.

The economic application uses this computational gain to revisit aggregate comovement after an uncertainty shock. In the baseline Bloom et al. (2018) model, output, investment, and employment fall, while consumption rises on impact. This lack of comovement arises because the decline in investment exceeds the decline in output, so goods-market clearing implies an increase in consumption. The authors discuss alternative household preferences, such as the GHH preference (Greenwood et al. (1988)), as a possible way to address this consumption response, but leave this extension unexplored because it requires an additional state variable and equilibrium price, which significantly increases the computational burden. The NN–KS makes it practical to examine this extension.

The results show that allowing for a finite Frisch elasticity, in contrast to the infinite elasticity assumed by Bloom et al. (2018), restores the comovement of aggregate variables following an uncertainty shock. Under an infinite Frisch elasticity, the real wage is determined solely by consumption through the household's labor supply condition, so declining employment does not exert additional downward pressure on wages. Consequently, the muted wage response leads to a

larger decline in employment. Because capital and labor are complementary in the Cobb–Douglas production function, this larger employment decline lowers the marginal product of capital, further reducing firms' incentive to invest and deepening the investment contraction. In contrast, a finite Frisch elasticity allows the real wage to fall as employment declines, cushioning the contraction in labor demand. The lower wage makes continued hiring more attractive, so the smaller decline in employment cushions the fall in the marginal product of capital and supports investment. Through goods-market clearing, the smaller reduction in investment spending allows consumption to fall alongside output.

Related Literature. The paper builds on the literature on solution methods for heterogeneous-agent models with aggregate uncertainty. Den Haan (2010b) compares solutions of the original KS model, and Terry (2017) examines heterogeneous-firm models. Alternatives include parameterized distributions (Algan et al., 2008), explicit aggregation (Den Haan and Rendahl, 2010), and methods combining distributional approximations with perturbation (Reiter, 2009; Winberry, 2018). More recent approaches also target the cost of finding equilibrium prices (Bakota, 2023; Lee, 2026). They propose alternative ways to mitigate this bottleneck by advancing the simulation without imposing market clearing in every period. My approach retains the standard KS simulation and its period-by-period market-clearing search, while removing repeated firm-level optimization from that search. Yang et al. (2025) adopt a similar strategy of treating prices as state variables, though they compute a Sequential Restricted Perceptions Equilibrium without the distribution included as a state variable. In contrast, my method retains the standard KS approximation to a rational-expectations equilibrium, using distributional moments in the forecasting rules and imposing market clearing in each simulation period.

The neural network extension is related to the growing literature on deep learning for dynamic economic models. My approach uses neural networks to approximate the price-conditioned policy function and value function, mitigating the curse of dimensionality while otherwise retaining the standard KS structure. However, enabling neural networks to handle (S, s)-type discontinuous policy functions is one of my contributions to this literature. This builds on the use of classification for discrete decisions in Maliar and Maliar (2022). Combining a discrete policy with conditional continuous policies accommodates discontinuous adjustment decisions while preserving exact inaction.

The application contributes to the literature on uncertainty shocks and aggregate comovement. Bloom et al. (2018) study how uncertainty interacts with capital and labor adjustment costs. Basu and Bundick (2017) obtain comovement through nominal price rigidity and countercyclical markups in a model of effective demand. The application here shows that a pure uncertainty shock can also generate simultaneous impact declines in output, consumption, investment, and employment under flexible prices, through household preferences and the general-equilibrium response to firms' adjustment decisions.

In Appendix C.4, I extend this analysis using GHH and quasi-separable preferences to examine how consumption–hours complementarity shapes aggregate comovement after an uncertainty shock, drawing on related work on preference specifications (Greenwood et al., 1988; Bilbiie et al., 2025).

Organization of the paper. Section 2 presents the model of Bloom et al. (2018) and explains the KS method and its simulation bottleneck. Section 3 develops policy precomputation, reports its computational gains, and explains the role of price errors in preserving accuracy. Section 4 presents NN–KS for the same model and evaluates its performance and accuracy. Section 5 studies household preferences and uncertainty-shock dynamics in the Bloom et al. (2018) model. Section 6 concludes. The Supplemental Appendix contains model specifications, implementation details, and additional results.

2 The Model and the Krusell–Smith Method

This section presents the model of Bloom et al. (2018) and explains how it is solved using the Krusell–Smith (KS) method. The model combines aggregate and idiosyncratic uncertainty with capital and labor adjustment costs. Its equilibrium prices must be determined jointly with firms' decisions, making the simulation stage of the KS method particularly costly.

2.1 Model

Time is discrete. A unit mass of heterogeneous firms produces a homogeneous good using a Cobb–Douglas technology in capital and labor:

$$y = z\varepsilon k^{\alpha} n^{\nu}. \tag{1}$$

Here, z is aggregate productivity, ε is idiosyncratic productivity, k is capital, and n is current employment.

Productivity processes. Both aggregate and idiosyncratic productivity follow persistent stochastic processes with time-varying volatility:

$$\log z' = \rho_z \log z + \sigma_z u'_z, \qquad u'_z \sim N(0,1), \tag{2}$$

$$\log \varepsilon' = \rho_\varepsilon \log \varepsilon + \sigma_\varepsilon u'_\varepsilon, \qquad u'_\varepsilon \sim N(0,1). \tag{3}$$

The time-varying volatilities σ_z and σ_ε capture switches between low-uncertainty and high-uncertainty regimes governed by a Markov process. These uncertainty states become part of the

aggregate state space. In the numerical implementation, a common uncertainty regime governs both productivity processes.

Capital dynamics and adjustment costs. Capital evolves according to the standard law of motion $k' = (1 - \delta_k)k + i$. Investment is subject to adjustment costs AC^k, which include a fixed cost proportional to output and a term capturing partial irreversibility:

$$\text{AC}^k = \mathbf{1}_{|i|>0}\, y\, F^K + S|i|\, \mathbf{1}_{i<0}. \tag{4}$$

Here, $\mathbf{1}_{(\cdot)}$ is an indicator function, F^K is the fixed disruption cost parameter, and S is the resale loss fraction for disinvested capital.

Labor dynamics and adjustment costs. Labor also faces adjustment frictions and evolves as

$$n = (1 - \delta_n)n_{-1} + s, \tag{5}$$

where n_{-1} is the previous period's employment, δ_n is an exogenous separation rate, and s represents net hiring or firing. Labor adjustment costs AC^n include a fixed cost proportional to output and a variable cost related to the hiring or firing flow:

$$\text{AC}^n = \mathbf{1}_{|s|>0}\, y\, F^L + |s|\, H\, w. \tag{6}$$

The parameter F^L is the fixed disruption cost for labor adjustment. The second term is a linear hiring or firing cost expressed as a fraction of the aggregate wage w. Both capital and labor adjustment costs absorb goods. Because labor adjustment depends on previous employment, n_{-1} is an additional endogenous state variable for the firm.

The firm's value function Let μ be the cross-sectional distribution of firms over their individual states (k, n_{-1}, ε) after current shocks are observed and before current choices are made, and $Z = (z, \sigma, \sigma_{-1})$. Following Bloom et al. (2018), I include σ_{-1} in the aggregate state vector. Firms choose investment i and hours n. The Bellman equation is

$$\begin{aligned} V(k, n_{-1}, \varepsilon; \mathrm{Z}, \mu) = \max_{i,n} \Big\{ & y - w(\mathrm{Z}, \mu)n - i - \text{AC}^k - \text{AC}^n \\ & + \mathbb{E}\left[m(\mathrm{Z}, \mu; \mathrm{Z}'\mu')V(k', n, \varepsilon'; \mathrm{Z}', \mu') \right] \Big\} \end{aligned} \tag{7}$$

given the law of motion for the distribution μ. $m(Z, \mu; Z'\mu')$ is the stochastic discount factor.

The household's problem and optimality conditions. A representative household consumes goods, supplies labor and owns shares in firms to maximize expected discounted utility. The

household's recursive problem is

$$W(\psi; Z, \mu) = \max_{C,N,\psi'} \Big\{ U(C,N) + \beta \mathbb{E}\big[W(\psi'; Z', \mu')\big] \Big\}, \tag{8}$$

where ψ denotes the household's portfolio of firm shares and Z collects the exogenous aggregate states.

The household is subject to the law of motion for μ and the sequential budget constraint

$$\begin{aligned} C + \int q(k', n, \varepsilon; Z, \mu)\, d\psi'(k', n, \varepsilon) \\ \leq w(Z, \mu) N + \int \rho(k, n_{-1}, \varepsilon; Z, \mu)\, d\psi(k, n_{-1}, \varepsilon), \end{aligned} \tag{9}$$

where q denotes the price of firm shares and ρ denotes the cum-dividend payoff on existing shares.

The household's period utility is given by

$$U(C,N) = \log C - \chi N, \qquad 0 < \beta < 1, \quad \chi > 0. \tag{10}$$

Consumption is the numeraire. Let p denote the Lagrange multiplier on the household's budget constraint. The household's first-order conditions imply

$$p = \frac{1}{C}, \qquad w = \frac{\chi}{p}, \qquad M_{t,t+1} = \beta \frac{p_{t+1}}{p_t}, \tag{11}$$

where $M_{t,t+1}$ is the stochastic discount factor used to value firm dividends.

The Bellman equation in marginal-utility units. We combine the household problem and the firm problem into one dynamic problem by using the household's optimality condition (11). Multiplying the firm's value in goods units, V, by marginal utility gives $\tilde{V} \equiv pV$. The Bellman equation in marginal-utility units is

$$\begin{aligned} \tilde{V}(k, n_{-1}, \varepsilon; Z, \mu) = \max_{i,n} \Big\{ & p(Z,\mu)\big[y - w(Z,\mu)n - i - \mathrm{AC}^k - \mathrm{AC}^n\big] \\ & + \beta \mathbb{E}\big[\tilde{V}(k', n, \varepsilon'; Z', \mu')\big] \Big\}. \end{aligned} \tag{12}$$

For a given firm policy, aggregate output, investment, employment, and adjustment costs are

$$\begin{aligned} Y &= \int z\varepsilon k^{\alpha} n^{\nu}\, d\mu, \qquad I = \int i\, d\mu, \\ N &= \int n\, d\mu, \qquad\quad AC = \int \big(\mathrm{AC}^k + \mathrm{AC}^n\big)\, d\mu. \end{aligned} \tag{13}$$

Goods-market clearing requires

$$C = Y - I - AC. \tag{14}$$

A recursive competitive equilibrium consists of value and policy functions, a price mapping, and a law of motion for the distribution such that firms solve (12), the household optimizes, and markets clear.

2.2 The Krusell–Smith Method

Following Krusell and Smith (1998), firms approximate the infinite-dimensional distribution μ with a finite set of moments. In the baseline model, the moment is aggregate capital,

$$K = \int k \, d\mu. \tag{15}$$

We use the two forecasting rules for K and p to compute the right-hand side of the Bellman equation (12). Let θ collect the parameters of the forecasting rules, then forecasting rules are

$$K' = \Gamma_K(Z, K; \theta), \tag{16}$$

$$\widehat{p} = \Gamma_p(Z, K; \theta), \qquad \widehat{w} = \chi / \widehat{p}. \tag{17}$$

For example, these rules can be specified as $\log K' = a_K(Z) + b_K(Z) \log K$ and $\log \widehat{p} = a_p(Z) + b_p(Z) \log K$.

The KS method iterates on the forecasting rules until they converge. Starting from an initial set of rules, each iteration proceeds in three steps: (1) solve the firm's Bellman equation under the current rules; (2) simulate the model and compare forecast values with realized outcomes; and (3) update the rules. The algorithm terminates when the change in the rules falls below a specified tolerance. I describe each step below.

Step 1: Solve the firm's problem under the forecasting rules. For fixed θ, solve

$$\begin{aligned} \tilde{V}_\theta(k, n_{-1}, \varepsilon; Z, K) = \max_{i,n} \Big\{ & \widehat{p}\big[y - \widehat{w}n - i - \mathrm{AC}^k - \mathrm{AC}^n(\widehat{w})\big] \\ & + \beta \mathbb{E}\big[\tilde{V}_\theta(k', n, \varepsilon'; Z', \Gamma_K(Z, K; \theta))\big] \Big\}. \end{aligned} \tag{18}$$

The value function is computed on a grid, using interpolation only for the aggregate capital. The current price is supplied by Γ_p, and future aggregate capital is supplied by Γ_K throughout this step.

The numerical implementation uses 91 points for capital, 37 for previous employment, 5 for idiosyncratic productivity, 5 for aggregate productivity, 2 for the current uncertainty regime, 2 for the lagged uncertainty regime, and 10 for aggregate capital:

$$91 \times 37 \times 5 \times 5 \times 2 \times 2 \times 10 = 3{,}367{,}000 \quad \text{state points.}$$

The capital and labor grids are constructed so that adjacent grid points correspond to depreciated

values. For example, $k_1 = (1-\delta_k)k_2$ for neighboring capital points, so moving from k_2 to k_1 represents exact inaction without paying the fixed cost.

Step 2: Simulate the economy and clear markets. In period t, the simulator observes (Z_t, μ_t) and computes $K_t = \int k\, d\mu_t$. Before updating the distribution, it must find the market-clearing price p_t. The forecast price $\widehat{p}_t = \Gamma_p(Z_t, K_t; \theta)$ need not equal the market-clearing price. The simulator therefore searches over candidate prices. For each candidate p, it computes firms' optimal policies, aggregates their choices using μ_t, obtains consumption from the goods-market-clearing condition (14), and checks the household's optimality condition. If the condition is not satisfied, it updates p and repeats the search.

Formally, fix the aggregate state Z, aggregate capital K, and a candidate price p. Since p may differ from the forecast $\widehat{p} = \Gamma_p(Z, K; \theta)$, the policy computed in Step 1 under the forecasting rules is generally not optimal at p. The standard KS method must therefore solve the maximization in equation (19) anew for every candidate price while the continuation value and forecasting rules remain fixed during this price search.

$$g_\theta(k, n_{-1}, \varepsilon; Z, K, p) \in \underset{(k', n) \in \mathcal{A}(k, n_{-1}, \varepsilon, Z)}{\arg\max} \Bigg\{ p\Big[y - \frac{\chi}{p} n - \big(k' - (1-\delta_k)k\big) - \mathrm{AC}^k - \mathrm{AC}^n(\chi/p)\Big] + \beta \mathbb{E}\big[\tilde{V}_\theta(k', n, \varepsilon'; Z', \Gamma_K(Z, K; \theta))\big] \Bigg\}, \tag{19}$$

where $\mathcal{A}(k, n_{-1}, \varepsilon, Z)$ is the feasible set of capital and employment choices.[2]

Using $g_\theta(k, n_{-1}, \varepsilon; Z, K, p)$ and the actual distribution μ, calculate aggregate quantities from (13) and implied consumption $C(p) = Y(p) - I(p) - AC(p)$. The current price must satisfy[3]

$$p = \frac{1}{C(p)}. \tag{20}$$

Once the price search is complete, the simulator updates the distribution using firms' decisions and realized shocks.

Step 3: Update the forecasting rules. After simulating the economy and discarding an initial transient, update θ using realized prices and aggregate capital. Repeat the solution and simulation steps until the forecasting rules converge, and evaluate their predictive accuracy using the simulated economy.

[2] For any fixed choice (k', n), the continuation term in (19) is independent of the candidate price. The price search therefore requires resolving the current maximization, but not recomputing the value function.

[3] Because fixed adjustment costs can generate jumps in excess demand, the original implementation evaluates excess demand on a grid of candidate prices and finds a zero of its interpolated excess demand.

The computational bottleneck. The main simulation cost arises in Step 2. For every candidate price, the standard method solves (19) for the individual states used in the simulation, aggregates the resulting decisions, and checks market clearing. A new price candidate requires another maximization for each individual state. This process starts again in every simulation period. In the Bloom et al. (2018) model, a 5,000-period simulation takes 290 minutes using a Fortran implementation with OpenMP parallelization on an AMD Ryzen 7 7700 processor. The next section removes this repeated firm-level optimization from the price search.

3 Accelerating the Krusell–Smith Method with Precomputation

The proposed method computes firms' responses to candidate prices before simulation. It retains the KS iteration on forecasting rules and the period-by-period search for market-clearing prices. Its computational gain comes from replacing the repeated maximization inside that search with evaluation of a precomputed policy.

3.1 Precomputing the Price-Conditioned Policy

After solving for $\tilde{V}_\theta$, construct a grid over individual states, aggregate states, aggregate capital, and candidate prices. At each grid point, solve

$$\bar{g}_\theta(k, n_{-1}, \varepsilon; Z, K, p) \in \underset{(k',n)\in\mathcal{A}(k,n_{-1},\varepsilon,Z)}{\arg\max} \Bigg\{ p\Big[y - \frac{\chi}{p}n - \big(k' - (1-\delta_k)k\big) - \mathrm{AC}^k - \mathrm{AC}^n(\chi/p)\Big] + \beta\mathbb{E}\big[\tilde{V}_\theta(k', n, \varepsilon'; Z', \Gamma_K(Z, K; \theta))\big] \Bigg\}, \tag{21}$$

Importantly, the candidate price p is not fixed at a single value; instead, the policy is computed at grid points spanning a range of candidate prices. The continuation value and forecasting rules are the same as in the KS method. The difference is when the conditional optimization is performed: optimal decisions are computed and stored before simulation begins. The policy table is then evaluated at each candidate price, using interpolation between grid points where needed.

During simulation, a trial price requires only evaluating the stored policy at $(k, n_{-1}, \varepsilon; Z_t, K_t, p)$ for each relevant individual state. Aggregate these decisions using the current distribution μ_t, evaluate the market-clearing residual, and update the trial price. Once the price search is complete, use the policy at that price to advance the distribution. The same precomputed table is used across price candidates and simulation periods. Whenever the outer iteration updates the forecasting rules, the Bellman equation is solved again, after which the precomputed policy table is recomputed.

The current price is an additional argument of the policy function because firms must respond to every price considered in the market-clearing search. A policy computed only at the forecast

Table 1: Computation time in the Bloom et al. (2018) model

Model	Method	Total	VFI	Precompute policy	Simulation
Bloom et al. (2018)	KS	6,141 min ($\approx$ 4 days)	7 min	–	290 min
Bloom et al. (2018)	Proposed	108 min	7 min	25 min	1.5 min

price $\Gamma_p(Z, K; \theta)$ would not provide this response. Including the candidate price directly makes it possible to move the conditional optimization out of the simulation loop.

3.2 Computation Time

Table 1 reports computation time for the baseline KS method and the proposed method. Total computation time falls from 6,141 minutes, approximately four days, to 108 minutes. The main source of the speedup is the simulation step: a 5,000-period simulation decreases from 290 minutes to 1.5 minutes because the proposed method avoids repeatedly solving the firm's maximization problem during the market-clearing price search. The value-function iteration takes 7 minutes in both methods, so the improvement does not come from solving the Bellman equation faster.

In this application, I use 25 grid points for p. Combining 3,367,000 state points with 25 price-grid points requires solving the firm problem (21) at 84,175,000 state–price combinations. Fixed adjustment costs further require a brute-force comparison across all discrete capital- and labor-adjustment regimes at each combination. These features account for the 25-minute precomputation time in the first outer iteration.

Subsequent iterations use the policy from the previous outer loop as a warm start, since small updates to the forecasting rules leave the previous policy close to optimal. This substantially reduces the cost of subsequent precomputation steps, bringing total computation time to 108 minutes. Thus, the upfront cost of constructing the policy table is more than offset by the reduction in simulation time.

3.3 Forecast-Deviation and Price-Level Formulations

Replacing repeated maximization with policy evaluation is the source of the simulation speedup. The accuracy of this approach depends on whether the precomputed policy captures firms' responses to prices. The placement of price-grid points is therefore crucial.[4]

A price grid centered on the forecasting rule. A natural approach is to precompute policies on a fixed grid of candidate price levels; I call this the *price-level formulation*. Such a grid must cover the full range of equilibrium prices across aggregate states. For example, in the Bloom et al. (2018) implementation, the realized equilibrium prices of 5,000 periods of simulation are within

[4]This issue is especially important in models in which optimal actions are sensitive to equilibrium prices, as discussed in Koby and Wolf (2020) and Winberry (2021).

the range from 1.1 to 1.7. As a result, constructing uniform grid points over the range can ensure that equilibrium prices must be found every period. However, this formulation is inefficient and can be inaccurate because many grid points may lie far from the actual equilibrium price relevant for a given aggregate state. Additionally, it leaves relatively coarse resolution near that price.

Instead, I construct a grid over deviations from the forecast price. I call this the *forecast-deviation formulation*. When the forecasting rule is informative, equilibrium prices tend to lie close to their forecasts, especially as the number of iterations for forecasting rules increases. Therefore, this grid concentrates points near the relevant prices and improves the accuracy of the market-clearing price.

Formally, define the deviation of the candidate price from its forecast as

$$p_{\text{error}} = p - \Gamma_p(Z, K; \theta). \tag{22}$$

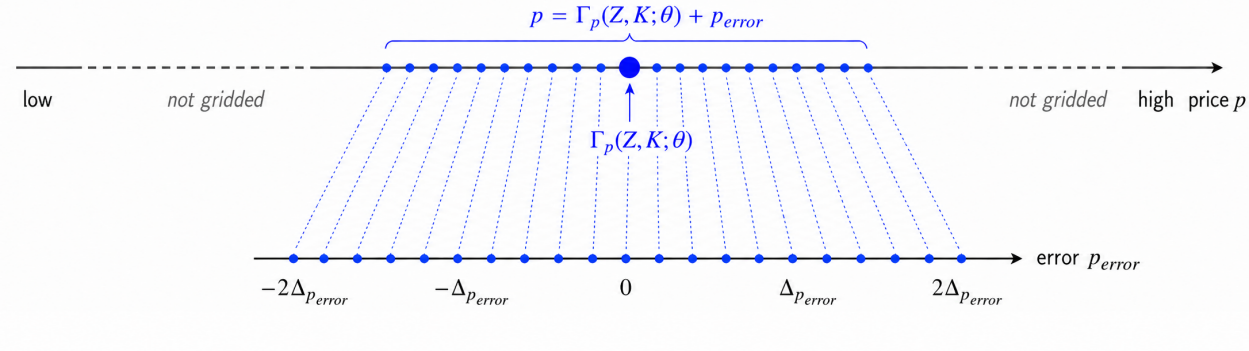
The price and wage corresponding to a deviation-grid point are

$$p = \Gamma_p(Z, K; \theta) + p_{\text{error}}, \qquad w = \frac{\chi}{\Gamma_p(Z, K; \theta) + p_{\text{error}}}. \tag{23}$$

The wage is recomputed consistently for every candidate p. During the search, p_{error} is a candidate-price deviation.

Figure 1 illustrates this construction. The lower axis places grid points around zero in the deviation p_{error}. The dotted lines map these points to candidate price levels on the upper axis according to $p = \Gamma_p(Z, K; \theta) + p_{\text{error}}$. The larger blue point marks the forecast itself. As the aggregate state changes, the price grid moves with the forecast, while the grid of deviations remains fixed.

Figure 1: Price grid centered on the forecast price



The portions of the upper axis without grid points show that the table covers a local range of price levels at each aggregate state. This concentration is important because the policy is precomputed only at discrete price nodes, whereas the market-clearing search generally evaluates candidate prices between those nodes. The accuracy of interpolation in the price dimension af-

fects the aggregate quantities used to determine the market-clearing price. With a given number of nodes, the forecast-deviation grid provides finer price resolution around the forecast and can reduce local interpolation error relative to a grid spanning the full range of price levels. [5]

Using these coordinates, the stored policy is represented as

$$\widetilde{g}_\theta(k, n_{-1}, \varepsilon; Z, K, p_{\text{error}}) = \bar{g}_\theta\big(k, n_{-1}, \varepsilon; Z, K, \Gamma_p(Z, K; \theta) + p_{\text{error}}\big). \tag{24}$$

Precomputation solves the same conditional optimization at the price implied by each deviation-grid point. During simulation, each trial price is converted to its deviation from the current forecast, and the policy is evaluated between the surrounding deviation-grid nodes. Precomputation eliminates repeated optimization, while the forecast-deviation grid helps preserve the accuracy of the policy evaluation needed during market clearing.

Implementation in the Bloom et al. (2018) price search. In the Bloom et al. (2018) model, fixed adjustment costs generate (S, s) policies, so excess demand can jump when a small price change induces a positive mass of firms to switch adjustment regimes. Their implementation therefore evaluates excess demand on a grid of candidate prices, constructs a continuous interpolated excess-demand schedule from these grid values, and searches for the price at which the interpolated schedule is zero.

The forecast-deviation formulation is useful in this procedure because it concentrates the price nodes around the forecast, where the market-clearing price is likely to lie. For a fixed number of nodes, this concentration narrows the price interval over which excess demand is interpolated near the solution and thereby improves the accuracy of the market-clearing price. Appendix A.2 uses the model of Khan and Thomas (2008) to compare the accuracy of the forecast-deviation and price-level formulations against the baseline KS method. Because the price search in this model does not interpolate excess demand, the comparison assesses the formulations directly. The forecast-deviation formulation nearly reproduces the baseline simulation paths and investment statistics, whereas the price-level formulation produces clear discrepancies.

3.4 Limitations of Precomputation

The gain during simulation comes at an upfront cost. If N_{state} denotes the number of state-grid combinations, N_e the number of price-deviation combinations, and d_a the number of stored policy outputs, precomputation requires approximately $N_{\text{state}} N_e$ conditional optimizations, and storage is proportional to $d_a N_{\text{state}} N_e$. These costs must be included when evaluating total runtime.

In the present model, the policy-function matrix requires roughly 1.25 GB of memory. Additional individual state variables, aggregate moments, or equilibrium prices multiply these storage

[5] The interval must still cover the candidates required by the search, including those encountered before the forecasting rules converge.

requirements and increase the cost of precomputation. For example, the preference extensions in Section 5 require an additional equilibrium price, the wage w, and an additional distributional moment, lagged aggregate employment N_{-1}. Relative to the baseline specification, these additions introduce two further dimensions into the policy table. These computational limitations motivate the neural-network approach developed in Section 4.

4 The NN–KS Method

In Section 3, the price-conditioned policy defined in (24) is stored as a table on a state–price grid. NN–KS approximates the same policy using neural networks—specifically, a discrete policy network g_d and a conditional policy network g_c. The economic problem, forecasting rules, market-clearing equation, distribution update, and outer KS iteration remain unchanged. The acceleration principle is also the same: firms' responses to candidate prices are learned before simulation, so the market-clearing search evaluates policies without repeatedly solving firms' optimization problems. Neural networks reduce the storage and interpolation costs of a large policy table, which contains 84,175,000 state–price combinations and occupies approximately 1.25 GB in the grid-based implementation.

Let $x_f = (k, n_{-1}, \varepsilon)$ denote a firm's individual state, let $Z = (z, \sigma, \sigma_{-1})$ denote the aggregate state, and let K denote aggregate capital. Throughout this section, the forecasting rules are $\Gamma_K(Z, K; \theta)$ and $\Gamma_p(Z, K; \theta)$. The policy networks take the price deviation p_{error} as an input; the candidate price and corresponding wage are recovered as

$$p = \Gamma_p(Z, K; \theta) + p_{\text{error}}, \qquad w = \chi / p,$$

as in (23). A value network approximates the Step 1 value function $\tilde{V}_\theta(x_f; Z, K)$ in (18). Its parameters ω_V, and the policy-network parameters ω_d and ω_c, are distinct from the forecasting-rule parameters θ. For each fixed θ, policy training covers the candidate-price range, whereas value updates use the forecast price, $p_{\text{error}} = 0$.

4.1 Decomposing the Single max: Discrete and Conditional Policies

Following and extending the classification approach of Maliar and Maliar (2022), I separate the choice of an adjustment regime from the conditional continuous choices of capital and employment. The discrete policy network g_d selects whether to adjust. Given this regime, the conditional policy network g_c determines (k', n) and imposes exact inaction in any dimension that is not adjusted.

Why separate discrete and continuous choices? An (S, s) policy is discontinuous: the optimal choice jumps when the firm crosses an adjustment threshold. A standard feedforward neural net-

work with continuous activation functions produces a continuous mapping and therefore cannot represent such a discontinuous policy exactly with a single output network.[6]

Exact inaction requires the network to output exactly $k' = (1-\delta_k)k$ for capital and $n = (1-\delta_n)n_{-1}$ for labor. Even a small approximation error produces nonzero investment or labor adjustment; it can therefore incorrectly trigger the fixed adjustment cost. Treating sufficiently small adjustments as zero would instead require an arbitrary numerical threshold. I avoid both problems by using two neural networks to represent the policy: a discrete network to choose the adjustment regime and a conditional continuous network to choose the action within that regime. The resulting composite policy can jump at adjustment boundaries and imposes exact inaction by construction.

Four adjustment regimes. Define $k_0 = (1-\delta_k)k$ and $n_0 = (1-\delta_n)n_{-1}$. With fixed k' and n, write the candidate return directly as a function of (k', n) for notational simplicity:

$$\begin{aligned} R_\theta(k', n) \equiv p\big[y - (\chi/p)n - (k' - k_0) - \mathrm{AC}^k - \mathrm{AC}^n(\chi/p)\big] \\ + \beta\mathbb{E}\big[\tilde{V}_\theta(k', n, \varepsilon'; Z', \Gamma_K(Z, K; \theta))\big]. \end{aligned} \tag{25}$$

Here and below, the common arguments $(x_f; Z, K, p_{\text{error}})$ are suppressed.

For numerical implementation, I decompose the maximization problem of (25) into four regimes:

$$\max_{k',n} R_\theta(k', n) = \max\big\{R^{00}_{\theta,\max}, R^{10}_{\theta,\max}, R^{01}_{\theta,\max}, R^{11}_{\theta,\max}\big\} = \max_{r\in\mathcal{R}} R^r_{\theta,\max}. \tag{26}$$

Here, $r = (r_k, r_n)$ belongs to $\mathcal{R} \equiv \{(0,0),(1,0),(0,1),(1,1)\}$. The first component r_k indicates capital adjustment, and the second component r_n indicates labor adjustment. In either component, 1 denotes adjustment and 0 denotes inaction; the superscripts 00, 10, 01, and 11 abbreviate these pairs.

For each $r \in \mathcal{R}$, let $R^r_\theta(k', n)$ denote the candidate return in (25) under regime r, and let $R^r_{\theta,\max}$ be its maximum subject to that regime's restrictions. Within each branch, the continuous choices maximize $R^r_\theta(k', n)$ subject to those restrictions:[7]

$$\begin{aligned} r=(0,0): &\quad k' = k_0, && n = n_0, && \text{(no capital or labor adjustment)}, \\ r=(1,0): &\quad k' \in \arg\max_{\tilde{k}'} R^{10}_\theta(\tilde{k}', n_0), && n = n_0, && \text{(capital adjustment only)}, \\ r=(0,1): &\quad k' = k_0, && n \in \arg\max_{\tilde{n}} R^{01}_\theta(k_0, \tilde{n}), && \text{(labor adjustment only)}, \\ r=(1,1): &\quad (k', n) \in \arg\max_{(\tilde{k}', \tilde{n})} R^{11}_\theta(\tilde{k}', \tilde{n}), && && \text{(capital and labor adjustment)}. \end{aligned} \tag{27}$$

[6] Classical uniform-approximation results apply to continuous target functions; see Cybenko (1989) and Hornik (1991).

[7] Mathematically, regime $(1,1)$ nests the other regimes because k' and n may equal k_0 and n_0, respectively. Under the continuous neural-network approximation, however, such exact equalities do not arise in practice.

Hence, the firm's single maximization separates into the outer discrete choice of the adjustment regime in (26), approximated by g_d, and the regime-specific choices in (27), approximated by g_c.

Discrete policy. The discrete policy network outputs four softmax scores:

$$\big(\pi_r(x_f; Z, K, p_{\text{error}})\big)_{r\in\mathcal{R}} = g_d(x_f; Z, K, p_{\text{error}}; \omega_d), \qquad \sum_{r\in\mathcal{R}} \pi_r = 1.$$

These normalized scores represent the network's predicted probabilities for the four adjustment regimes. The actual regime is selected deterministically by

$$\widehat{r}(x_f; Z, K, p_{\text{error}}) = \arg\max_{r\in\mathcal{R}} \pi_r(x_f; Z, K, p_{\text{error}}).$$

Conditional continuous policy. The conditional policy network receives the same state and price deviation, together with the selected regime encoded by its two binary digits:

$$(k', n) = g_c(x_f; Z, K, p_{\text{error}}, r; \omega_c).$$

The regime restrictions impose $k' = k_0$ in regimes 00 and 01. Likewise, they impose $n = n_0$ in regimes 00 and 10 .

Figure 2: Neural network architectures for discrete and conditional policies.

(a) Discrete policy network g_d (b) Conditional policy network g_c

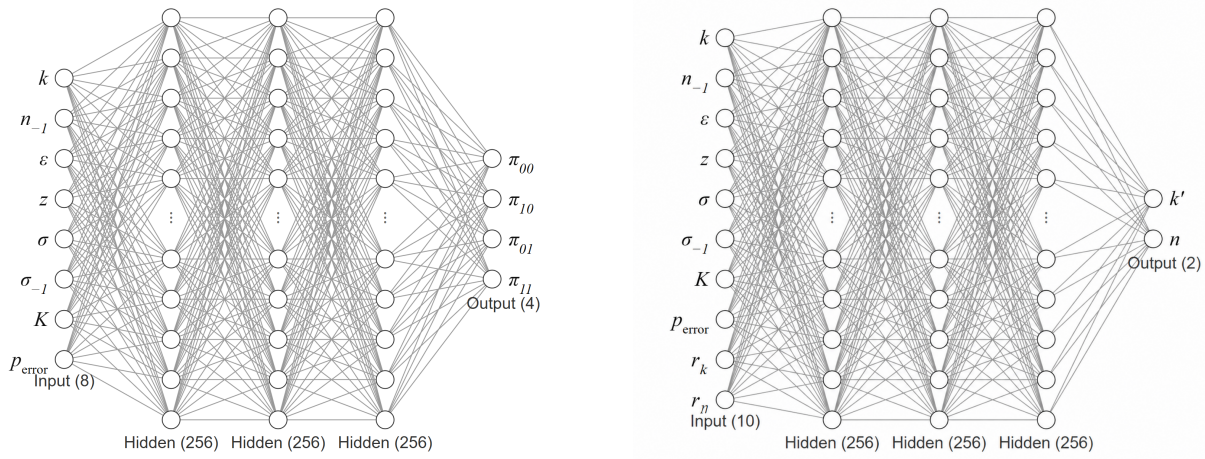


Figure 2 shows the two networks. Their common inputs are $(k, n_{-1}, \varepsilon, z, \sigma, \sigma_{-1}, K, p_{\text{error}})$. The discrete network selects a regime, and the conditional network supplies capital and employment choices within it. Simulation requires only these forward evaluations at each candidate price.

Training the discrete policy. For fixed θ, I train g_d as a supervised classifier following Maliar and Maliar (2022). For each training observation j, the current conditional-policy and value ap-

proximations imply a return for each regime. I assign the highest-return regime as the target label:

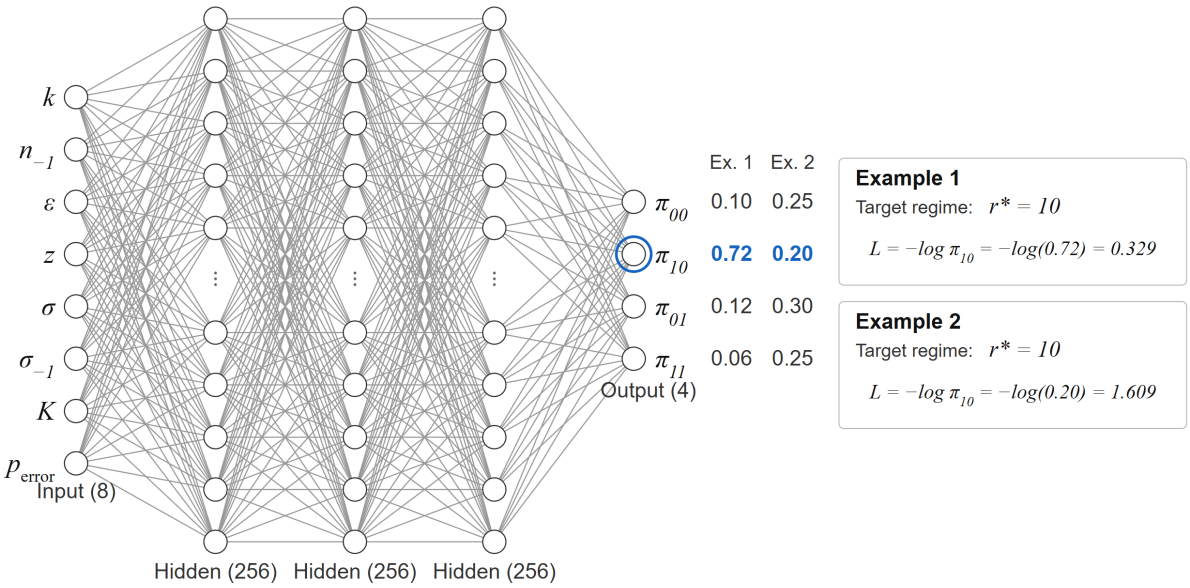
$$r_j^*(x_{f,j}; Z_j, K_j, p_{\text{error},j}) \in \arg\max_{r \in \mathcal{R}} R^r_{\theta,\max}(x_{f,j}; Z_j, K_j, p_{\text{error},j}).$$

For a batch of B observations indexed by j, I train the network to assign high probability to each target regime by minimizing the cross-entropy loss

$$\mathcal{L}_{\text{disc}}(\omega_d) = -\frac{1}{B}\sum_{j=1}^{B} \log \pi_{r_j^*}(x_{f,j}; Z_j, K_j, p_{\text{error},j}).$$

Figure 3 illustrates the loss for two observations with target regime $r_j^* = 10$. When g_d assigns probability $\pi_{10} = 0.72$, the loss is $-\log(0.72) \approx 0.329$; when it assigns $\pi_{10} = 0.20$, the loss rises to $-\log(0.20) \approx 1.609$. Thus, the loss penalizes low probability on the highest-return regime and trains g_d to reproduce the firm's discrete choice.

Figure 3: Training the discrete policy network



Training the conditional policy. For each sampled state and price deviation, let $\widehat{r}_j$ be the regime selected by g_d, and set

$$(k'_j, n_j) = g_c(x_{f,j}; Z_j, K_j, p_{\text{error},j}, \widehat{r}_j; \omega_c), \qquad i_j = k'_j - (1-\delta_k)k_j.$$

Holding the value network and discrete choices fixed, update ω_c to maximize the return at these choices, or equivalently minimize

$$\mathcal{L}_{\text{cont}}(\omega_c) = -\frac{1}{B}\sum_{j=1}^{B} R_\theta\Big(x_{f,j}, (k'_j, n_j); Z_j, K_j, \Gamma_p(Z_j, K_j; \theta) + p_{\text{error},j}\Big).$$

The continuation value uses the value network. Training focuses on the predicted regimes.

Value network. The value network satisfies

$$\tilde{V}_{\text{nn}}(x_f; Z, K; \omega_V) \approx \tilde{V}_\theta(x_f; Z, K).$$

The value-network target is the Bellman return evaluated at the forecast price, as in Step 1 of Section 2.2. For each sampled state, set

$$\widehat{p}_j = \Gamma_p(Z_j, K_j; \theta),$$

and evaluate the composite policy at $p_{\text{error},j} = 0$:

$$\widehat{r}_j = \arg\max_{r \in \mathcal{R}} \pi_r(x_{f,j}; Z_j, K_j, 0), \qquad (k'_j, n_j) = g_c(x_{f,j}; Z_j, K_j, 0, \widehat{r}_j; \omega_c).$$

The target for the value network is R_θ evaluated at the selected action and forecast price. The expectation uses the model's conditional shock transitions, with next-period aggregate capital forecast by $\Gamma_K(Z_j, K_j; \theta)$:[8]

$$\mathcal{L}_{\text{val}}(\omega_V) = \frac{1}{B}\sum_{j=1}^{B}\Big[\tilde{V}_{\text{nn}}(x_{f,j}; Z_j, K_j; \omega_V) - R_\theta\Big(x_{f,j}, (k'_j, n_j); Z_j, K_j, \widehat{p}_j\Big)\Big]^2.$$

The value network minimizes this squared Bellman error.

Alternating training. For fixed forecasting rules, training alternates among the discrete policy, conditional policy, and value networks. At each update, I train one network while holding the other two fixed: g_d is updated with g_c and the value network fixed, g_c is updated with g_d and the value network fixed, and the value network is updated with both policy networks fixed. I repeat this cycle until convergence. Numerical settings are reported in Appendix B.1.

Once training is complete, NN–KS proceeds as in the grid-based method. The trained networks are used to simulate the economy, and the simulated prices and aggregate capital are then used to update the forecasting rules. The networks are retrained under the updated forecasting rules, and this cycle of training, simulation, and forecasting-rule updates is repeated until con-

[8] When training the value network, the continuation value in this target is evaluated using a target network.

vergence. NN–KS therefore changes the representation and numerical solution of the value and price-conditioned policy functions while retaining the economic problem and simulation structure of Sections 2 and 3.

4.2 Results

I evaluate NN–KS by comparing NN–KS with the KS benchmark. The computations were carried out on a system equipped with an AMD Ryzen 7 7700 CPU and an NVIDIA GeForce RTX 4060 Ti GPU. The results are mostly based on single precision (FP32). [9] It is important to note that the benchmark results for Bloom et al. (2018) reported here are based on a slightly modified version of their original code, with a modification to the way the distribution is updated during simulation. Consequently, these results may differ from those reported in the published version of Bloom et al. (2018). [10]

Table 2: Speed comparison for the Bloom et al. (2018) model

Method	Simulation Time (min)	Total Time (min, 20 outer loops)
KS	290	6,141 ($\approx$ 4 days)
NN–KS	3	183

Note: Simulation time is measured for 5,000 periods during the first simulation. Total time covers 20 outer loops.

Computation and Simulation Speed. Table 2 presents the simulation and total computation times for each method. NN–KS reduces the total computation time from 6,141 minutes (approximately four days) to 183 minutes. A 5,000-period simulation falls from 290 minutes to 3 minutes.[11] The exact total time depends on the initial coefficients of the forecasting rules; the reported results are based on 20 outer-loop iterations using the original initial values.

Training in NN–KS proceeds cumulatively across the outer loops. Updating the forecasting rules changes the problem faced by the networks, but the change is gradual, so the networks obtained in the previous outer loop provide useful initial conditions for the next one. NN–KS therefore does not relearn the solution from scratch in every outer loop. Instead, it progressively reduces the approximation error as the forecasting rules converge; after the first outer loop, approximately 3–4 minutes of additional training is typically sufficient before the next simulation.

[9] The code is implemented primarily in single precision (FP32), with double precision used only for updating the distribution and aggregating variables in the simulation.

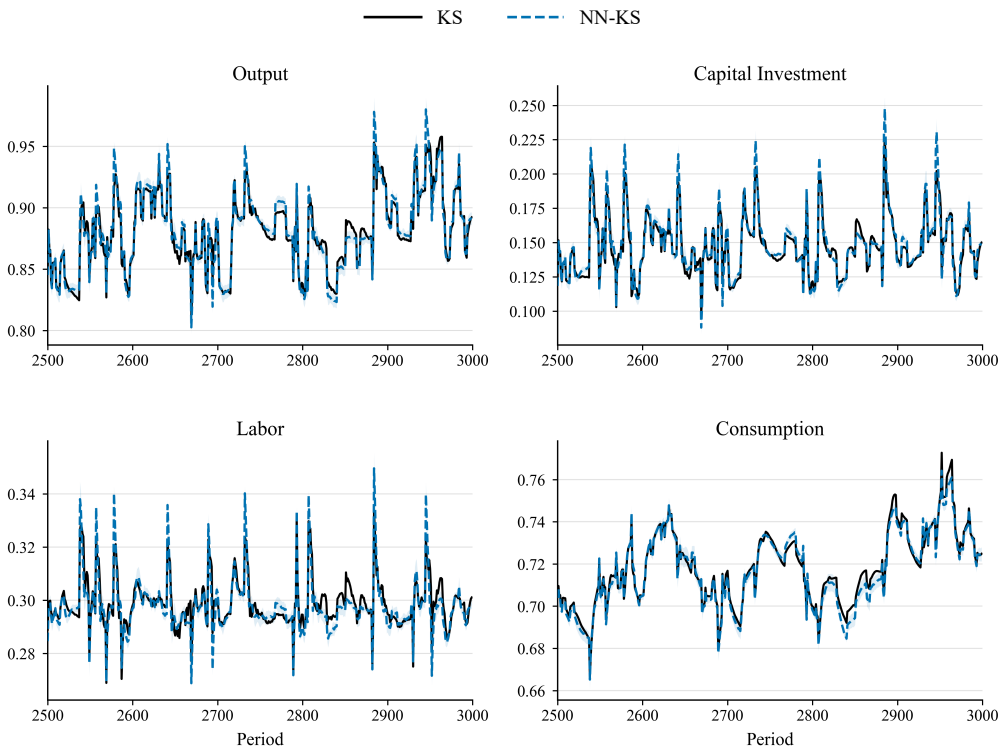
[10] In their original code, they set a threshold of 10^{-4} on the mass of firms in the distribution grid to determine which points to update. They ignore points with a mass below the threshold to reduce computation time. In the modified version, I update all points in the distribution grid regardless of their mass, and the results reported in this paper are based on this modified version. One could replicate the result by setting `disttol` = 0.0 and `GEerrorswitch` = 2 in their code.

[11] When the NN–KS simulation is run on the CPU alone, the same 5,000-period simulation takes approximately 38 minutes rather than 3 minutes on the GPU.

For the present Bloom et al. (2018) application, however, the state space is still small enough for conventional grid-based value-function iteration. The accelerated grid-based method completes the same 20 outer loops in 108 minutes (Table 1), compared with 183 minutes for NN–KS. The grid-based solution is also more accurate in this case. As shown in Appendix B.2.1, its mean relative Bellman error is 3.69×10^{-7} at grid nodes and 5.45×10^{-6} at off-grid states, compared with approximately 3.22×10^{-4} for NN–KS in both samples. Thus, when a sufficiently fine grid remains feasible, the grid-based method is both faster and more accurate. The advantage of NN–KS emerges in larger models, where additional individual states, aggregate moments, or equilibrium prices make the storage and interpolation requirements of a grid-based policy table prohibitively large.

Simulation Paths and Business-Cycle Properties. Figure 4 reports unconditional simulation paths for aggregate variables over periods 2500–3000. Table 3 reports the corresponding HP-filtered business-cycle statistics.

Figure 4: Simulation paths generated by KS and NN–KS



Note: The figure reports simulation paths for periods 2500–3000 using the same realization of aggregate and stochastic volatility shocks. The solid black line represents KS, while the dashed blue line represents the arithmetic mean of NN–KS across five independent runs. The shaded area indicates plus or minus one across-run standard deviation. The panels report output, capital investment, labor, and consumption.

Figure 4 and Table 3 provide a consistent comparison. The NN–KS paths closely track the

Table 3: HP-filtered business-cycle statistics

	Output	Investment	Labor	Consumption
Standard deviations (%)				
KS	1.8790	9.5546	2.1926	0.8673
NN–KS	2.0649	10.4607	2.5838	0.8836
(run s.d.)	0.0321	0.2580	0.0884	0.0177
Ratio to output				
KS	1.0000	5.0850	1.1669	0.4616
NN–KS	1.0000	5.0658	1.2512	0.4279
(run s.d.)	0.0000	0.0896	0.0379	0.0042
Correlations with output				
KS	1.0000	0.9440	0.7996	0.3303
NN–KS	1.0000	0.9521	0.8372	0.2844
(run s.d.)	0.0000	0.0031	0.0105	0.0439

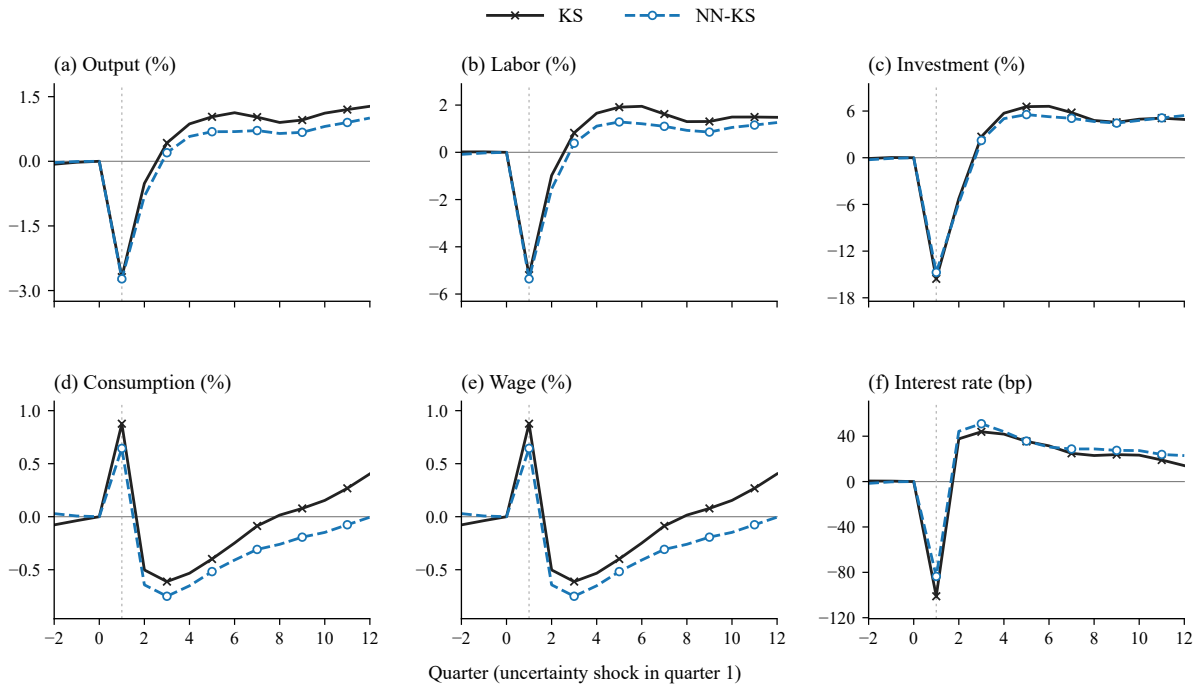
Note: Statistics are based on the same realization of aggregate and stochastic volatility shocks, discarding the first 500 periods. Each series is expressed in logarithms, the HP filter with smoothing parameter $\lambda = 1600$ is applied, and the cyclical component is multiplied by 100 to express deviations in percent. NN–KS statistics are reported as averages over 5 independent runs. The third row in each block ("run s.d.") reports the across-run standard deviation computed from these 5 runs.

timing and overall movements of the KS benchmark, but the figure shows somewhat larger fluctuations in capital investment and labor. The business-cycle statistics quantify this difference: the standard deviation of capital investment rises from 9.5546 under KS to 10.4607 under NN–KS, while that of labor rises from 2.1926 to 2.5838. One possible explanation for this difference is that NN–KS allows continuous adjustment magnitudes, whereas the benchmark KS implementation restricts choices to a discrete grid without interpolating between candidate actions. This greater flexibility can make adjustment more attractive relative to inaction for firms near adjustment thresholds, potentially contributing to larger aggregate fluctuations. Nevertheless, the investment-to-output volatility ratio is nearly identical across the two methods, and the correlations of investment and labor with output remain close. Thus, NN–KS reproduces the broad simulation dynamics, although it somewhat overstates the volatility of capital investment and labor.

Impulse Responses Figure 5 compares uncertainty-shock responses from KS and NN–KS under the original preference specification. Both methods generate a sharp decline followed by a rebound in output, employment, and investment. Although the rebound paths differ somewhat, the overall dynamics and response magnitudes are broadly similar. Output falls by 2.69% under KS and 2.73% under NN–KS on impact; investment falls by 15.56% and 14.74%, respectively. Consumption and the wage rise under both methods. The real risk-free rate also falls on impact, by 101.1 basis points under KS and 83.6 basis points under NN–KS.

Accuracy of the Forecasting Rules. Table 4 presents the internal accuracy of the forecasting rules for the aggregate price p and next-period aggregate capital K'. I report the Den Haan statistics

Figure 5: Uncertainty-shock responses in the original model: KS and NN–KS



Notes: The uncertainty shock occurs in quarter 1. Panels (a)–(e) show $100 \log(\overline{X}_t / \overline{X}_0)$, using arithmetic mean levels across 2,000 economies per method. Panel (f) shows $10^4(\bar{r}^f_t - \bar{r}^f_0)$, where $r^f_t = [\mathbb{E}_t(\beta p_{t+1}/p_t)]^{-1} - 1$; rates are quarterly and not annualized. The conditional expectation uses each method's aggregate forecasting rules. Each series is normalized to its own pre-shock mean.

(maximum and mean), the root mean squared error (RMSE), and the R^2 from forecast regressions. The two methods produce comparable forecast accuracy. For p, NN–KS has lower maximum and mean Den Haan errors, a slightly higher RMSE, and a slightly lower R^2. For K', NN–KS has a slightly higher RMSE and a higher R^2, while its maximum Den Haan error is slightly higher and its mean error is slightly lower. Appendix B.2.2 provides detailed forecast accuracy statistics broken down by aggregate shock state (see Table 9).

One possible explanation for the R^2 values below the approximately 0.99 typically reported in the literature is the (S, s)-type adjustment behavior of firms at distribution grid points with small probability masses. Retaining these grid points may make aggregate dynamics more difficult to forecast. The original implementation of Bloom et al. (2018) omits grid points with masses below 10^{-4} when updating the distribution. This truncation may help explain the higher R^2 of approximately 0.99 reported in their study. [12] Extending the simulation to 10,000 periods yields only a slight improvement in forecast accuracy.

[12] Applying the same truncation threshold can distort NN–KS simulations. Because NN–KS permits continuous choices, the distribution update distributes the mass associated with each choice across four neighboring grid points. In this implementation, the mass per grid point tends to be approximately one-quarter of that in the original implementation, making the same truncation threshold more restrictive.

Table 4: Accuracy of the forecasting rules

	p (%)		K' (%)	
	KS	NN-KS	KS	NN-KS
Den Haan Statistics				
Maximum	3.80	3.59	5.68	5.77
Mean	0.96	0.82	1.83	1.81
Root Mean Squared Error				
RMSE	0.43	0.46	0.25	0.27
Forecast Regression R^2				
R^2	0.9793	0.9747	0.9957	0.9958

Note: The table reports Den Haan statistics (maximum and mean), root mean squared error (RMSE, %), and R^2 from forecast regressions, comparing the KS benchmark and the NN-KS method.

5 Household Preferences and Comovement after an Uncertainty Shock in Bloom et al. (2018)

This section examines how household preferences shape the general-equilibrium transmission of an uncertainty shock in Bloom et al. (2018). I first use the original-model impulse responses in in Figure 5 to explain the failure of comovement. I then introduce separable preferences with a finite Frisch elasticity, describe the equilibrium conditions and their NN–KS implementation, and examine the resulting impulse responses. Appendix C.4 extends the analysis by retaining a finite Frisch elasticity and varying consumption–hours complementarity to study the allocation between consumption and investment.

5.1 The Original Specification and the Failure of Comovement

An uncertainty shock raises the option value of postponing costly capital and labor adjustments. Firms therefore adopt a wait-and-see response: hiring, investment, and output decline even though aggregate productivity itself does not change. Figure 5, presented in Section 4, shows this response under the original preferences for both KS and NN–KS. Under NN–KS, output falls by approximately 2.7% and employment by 5.4% on impact, while investment falls by approximately 14.7%. Consumption instead rises by approximately 0.64%. Thus, the original specification does not generate simultaneous declines in output, consumption, investment, and employment, and this failure is common to both solution methods.

The goods-market-clearing condition helps interpret the consumption response:

$$C_t = Y_t - I_t - AC_t, \tag{28}$$

where AC_t denotes aggregate capital and labor adjustment costs in units of goods. Consumption rises when the reduction in investment and adjustment spending exceeds the loss of output in goods units. The positive consumption response in Figure 5 therefore reflects the resources released by the contraction in these expenditures.

In the original specification, household preferences are $U(C, N) = \log C - \chi N$, with an infinite Frisch elasticity. As in Section 2.1, the marginal-utility price and real wage satisfy

$$p_t = U_C(C_t, N_t) = \frac{1}{C_t}, \qquad w_t = -\frac{U_N(C_t, N_t)}{U_C(C_t, N_t)} = \chi C_t = \frac{\chi}{p_t}. \tag{29}$$

Under the original specification, the household's labor supply schedule is horizontal conditional on consumption, reflecting an infinite Frisch elasticity. When an uncertainty shock reduces firms' demand for labor, declining employment does not itself induce a reduction in the real wage. Consequently, the contraction in labor demand is absorbed primarily through a reduction in employment rather than a lower wage. Since capital and labor are complementary in production, the larger employment contraction reduces the marginal product of capital and further weakens firms' incentives to invest. This reinforces the initial wait-and-see response, resulting in substantial declines in employment and investment.

The mechanism can also be understood through goods-market clearing. The sharp contraction in investment and adjustment expenditures releases more resources than are lost through declining output. At a given candidate price, this can create an excess supply of goods available for consumption. To restore market clearing, the marginal-utility price adjusts downward. A lower p_t raises household consumption through $C_t = 1/p_t$, while also encouraging firms to partially recover their investment from the initial contraction. Both responses help absorb the excess supply of goods. Nevertheless, investment remains below its pre-shock level, and the equilibrium is characterized by higher consumption despite lower output, employment, and investment. Thus, the absence of an employment-dependent downward wage adjustment contributes to the failure of comovement, motivating the introduction of a finite Frisch elasticity.

5.2 Finite Frisch Elasticity: Model, Equilibrium, and Computation

Household preferences and equilibrium. I retain the production technology, productivity processes, capital and labor adjustment costs, and firm state $x_f = (k, n_{-1}, \varepsilon)$ from Section 2. The only change is to the curvature of labor disutility. The household maximizes expected discounted utility with period utility

$$U(C, N) = \log C - \chi \frac{N^{1+\phi}}{1+\phi}, \qquad \phi > 0,$$

so the Frisch elasticity is $1/\phi$. The household price conditions become

$$p_t = \frac{1}{C_t}, \qquad w_t = \chi C_t N_t^{\phi}, \qquad M_{t,t+1} = \beta \frac{p_{t+1}}{p_t}. \tag{30}$$

For each elasticity, I recalibrate the labor-disutility weight using $\chi = \bar{w}/(\bar{C}\bar{N}^{\phi})$ at the common reference allocation $(\bar{C}, \bar{N}, \bar{w}) = (0.73, 0.299, 1.451)$, leaving all other structural parameters unchanged.

Unlike the original relation $w_t = \chi / p_t$, equation (30) makes the wage depend on equilibrium employment. Thus, w_t cannot be inferred from p_t alone; the equilibrium price, wage, and firm choices must be determined jointly. For each candidate pair (p, w) and the current distribution μ_t, the firm policies imply output $Y_t(p, w)$, investment $I_t(p, w)$, adjustment spending $AC_t(p, w)$, and employment $N_t(p, w)$. Let $C_t(p, w) = Y_t(p, w) - I_t(p, w) - AC_t(p, w)$. Goods- and labor-market clearing require

$$p - \frac{1}{C_t(p, w)} = 0, \qquad w - \chi C_t(p, w) N_t(p, w)^{\phi} = 0. \tag{31}$$

Once these residuals vanish, the associated policies determine current allocations and the next-period distribution in the same way as in the original model.

NN–KS implementation. To approximate the distributional state, I retain aggregate capital K_t and add lagged aggregate employment N_{t-1}, which helps forecast the wage. With $Z_t = (z_t, \sigma_t, \sigma_{t-1})$, the forecasting rules are

$$\begin{aligned} K_{t+1} &= \Gamma_K(Z_t, K_t, N_{t-1}; \theta), \\ \widehat{p}_t &= \Gamma_p(Z_t, K_t, N_{t-1}; \theta), \qquad \widehat{w}_t = \Gamma_w(Z_t, K_t, N_{t-1}; \theta). \end{aligned} \tag{32}$$

Current aggregate employment becomes a distributional moment next period. In the continuation value, its forecast is recovered from the household conditions as

$$\widehat{N}_t = \left(\frac{\widehat{p}_t \widehat{w}_t}{\chi} \right)^{1/\phi}. \tag{33}$$

Thus, the value network takes (x_f, Z, K, N_{-1}) as inputs, and its continuation value uses next-period individual states (k', n, ε'), next-period aggregate shocks, forecast capital, and $\widehat{N}_t$.

The discrete and conditional policy networks additionally take both candidate prices, represented by the deviations $p_{\text{error}} = p - \widehat{p}$ and $w_{\text{error}} = w - \widehat{w}$. For fixed forecasting rules, the policies are trained over both price deviations. The continuation state is also evaluated at the forecasts, so candidate current prices change the current payoff without changing the aggregate continuation state. During simulation, each candidate pair requires only policy-network evaluations and aggregation. A two-dimensional price search solves equation (31), the distribution is updated, and the simulated data are used to revise the forecasting rules in the outer KS iteration. This extends

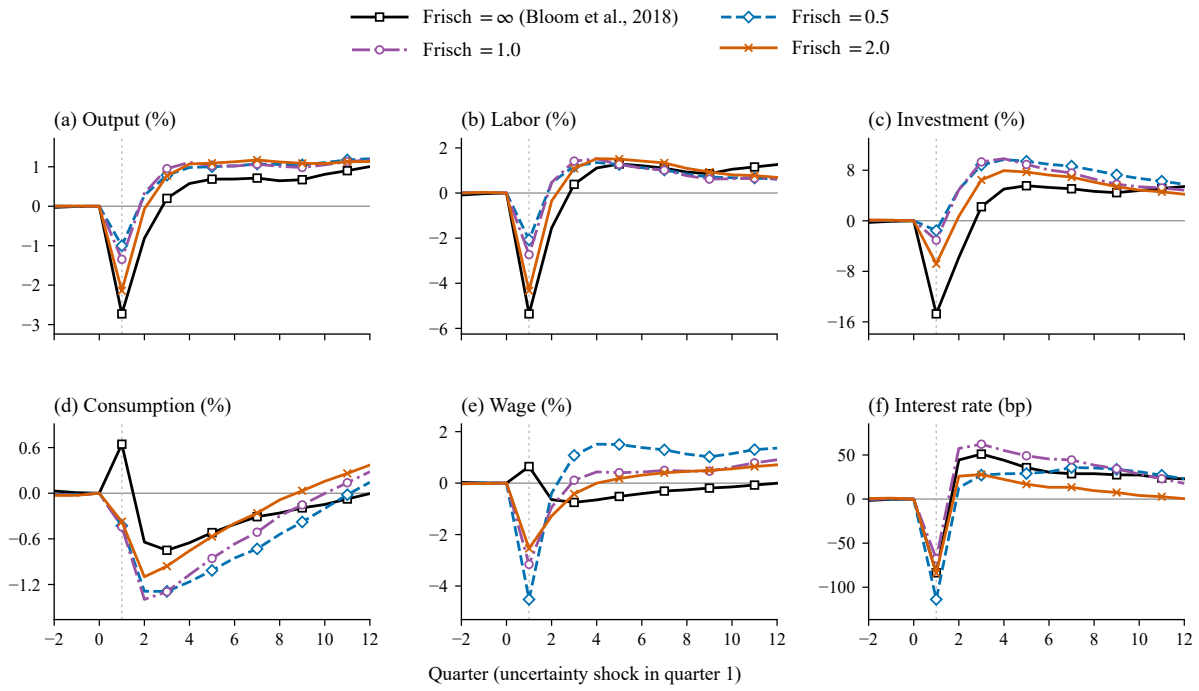
the precomputation principle of Section 4 to two equilibrium prices. Appendix C.1 provides the full network inputs and training details, including the extension to the preferences considered in Appendix C.4.

Computational cost. The preference extension adds only modest computational cost. NN–KS completes 20 outer-loop iterations in about 200 minutes, compared with 183 minutes for the original Bloom et al. (2018) specification. A 5,000-period simulation takes approximately 3.5 minutes, versus about 3 minutes in the original model. Thus, jointly forecasting the wage and searching over two equilibrium prices does not substantially increase computation time. Appendix B.2.3 reports the accuracy of the three forecasting rules.

5.3 Finite Frisch Elasticity and the Wage Channel

Figure 6 compares the original specification with separable preferences at Frisch elasticities of 0.5, 1, and 2. All four cases retain log utility from consumption and have no consumption–hours complementarity. Making the Frisch elasticity finite reverses the impact consumption response: output, consumption, investment, and employment now all decline.

Figure 6: Uncertainty-shock responses: original and finite Frisch elasticities



Notes: The shock occurs in quarter 1. Panels (a)–(e) show $100 \log(\overline{X}_t / \overline{X}_0)$, with arithmetic means across 2,000 NN–KS economies per specification. Panel (f) shows the change in the quarterly real risk-free rate in basis points, computed as $r_t^f = [\mathbb{E}_t(\beta p_{t+1}/p_t)]^{-1} - 1$; it is not annualized. All cases retain separability and log consumption utility. The infinite-Frisch benchmark corresponds to the original model in Figure 5. The finite-Frisch cases share shock draws; the benchmark uses independent draws.

The household wage condition in equation (30) implies, within each economy,

$$\Delta \log w_t = \Delta \log C_t + \phi \Delta \log N_t. \tag{34}$$

With $\phi > 0$, declining employment puts direct downward pressure on the wage. At a Frisch elasticity of 2, the wage falls by 2.54%, employment by 4.33%, and investment by 6.81% on impact; output and consumption decline by 2.14% and 0.37%, respectively. The quarterly real risk-free rate falls by 83.9 basis points.

The lower wage makes continued employment more attractive. The smaller employment contraction is consistent with a weaker investment contraction through capital–labor complementarity: preserving employment cushions the fall in the marginal product of capital. Through the goods-market-clearing condition (28), the balance between the output loss and lower investment and adjustment spending allows consumption to decline along with output. A finite Frisch elasticity therefore generates the missing comovement without an adverse first-moment shock.

As the Frisch elasticity falls from 2 to 1 and 0.5, the impact wage decline increases from 2.54% to 3.16% and 4.52%, while the employment decline decreases from 4.33% to 2.73% and 2.07%. Investment declines by 6.81%, 3.06%, and 1.54%, respectively, and consumption falls in all three cases. A smaller Frisch elasticity strengthens the wage adjustment and moderates the contractions in employment, output, and investment. I use Frisch $= 2$ because it generates labor business-cycle moments closest to the data among these three calibrations, as reported in Appendix C.2. Appendix C.3 examines how the wage channel depends on labor adjustment costs. Appendix C.4 examines how consumption–hours complementarity affects the aggregate dynamics using the GHH preference (Greenwood et al., 1988) and the quasi-separable utility (Bilbiie et al., 2025).

6 Conclusion

This paper develops a method that substantially accelerates the Krusell–Smith method while preserving its familiar structure. The key idea is to precompute a price-conditioned policy function before simulation, so that each candidate price can be evaluated without resolving the firm's optimization problem. Expressing the candidate price as a deviation from its forecast also concentrates the approximation on the region relevant to the market-clearing search.

In the Bloom et al. (2018) model, which features fixed adjustment costs and discontinuous policies, computation time falls from 6,141 minutes to 108 minutes. To overcome the scalability limits of the grid-based approach, I extend the method using neural networks and a discrete–continuous representation of (S, s)-type policies. This NN–KS extension completes in 183 minutes and broadly reproduces the KS results. The same approach can be applied to other heterogeneous-agent models with implicit market-clearing prices, including models with multiple prices.

The economic application shows that the model generates impact comovement after a pure un-

certainty shock when the Frisch elasticity is finite. As employment falls, the wage declines, cushioning the contractions in employment and investment; the smaller investment decline allows consumption to fall alongside output. Consumption–hours complementarity is not necessary for impact comovement. Appendix C.4 shows how complementarity affects the consumption–investment mix through marginal utility and, in turn, the real risk-free rate; under GHH preferences, strong complementarity raises investment on impact.

References

Ahn, S., G. Kaplan, B. Moll, T. Winberry, and C. Wolf (2018). When inequality matters for macro and macro matters for inequality. *NBER macroeconomics annual 32*(1), 1–75.

Alfaro, I., N. Bloom, and X. Lin (2024). The finance uncertainty multiplier. *Journal of Political Economy 132*(2), 577–615.

Algan, Y., O. Allais, and W. J. Den Haan (2008). Solving heterogeneous-agent models with parameterized cross-sectional distributions. *Journal of Economic Dynamics and Control 32*(3), 875–908.

Algan, Y., O. Allais, W. J. Den Haan, and P. Rendahl (2014). Solving and simulating models with heterogeneous agents and aggregate uncertainty. *Handbook of computational economics 3*, 277–324.

Auclert, A., B. Bardóczy, M. Rognlie, and L. Straub (2021). Using the sequence-space jacobian to solve and estimate heterogeneous-agent models. *Econometrica 89*(5), 2375–2408.

Azinovic, M., L. Gaegauf, and S. Scheidegger (2022). Deep equilibrium nets. *International Economic Review 63*(4), 1471–1525.

Azinovic-Yang, M. and J. Žemlička (2025). Deep learning in the sequence space. *arXiv preprint arXiv:2509.13623*.

Bakota, I. (2023). Market clearing and krusell-smith algorithm in an economy with multiple assets. *Computational Economics 62*(3), 1007–1045.

Basu, S. and B. Bundick (2017). Uncertainty shocks in a model of effective demand. *Econometrica 85*(3), 937–958.

Bayer, C., R. Luetticke, L. Pham-Dao, and V. Tjaden (2019). Precautionary savings, illiquid assets, and the aggregate consequences of shocks to household income risk. *Econometrica 87*(1), 255–290.

Bilbiie, F., F. Hanks, and S. Lavender (2025). Complementarity, heterogeneity, and multipliers: Utility for hank. CEPR Discussion Paper 20804, Centre for Economic Policy Research.

Bloom, N., M. Floetotto, N. Jaimovich, I. Saporta-Eksten, and S. J. Terry (2018). Really uncertain business cycles. *Econometrica 86*(3), 1031–1065.

Boppart, T., P. Krusell, and K. Mitman (2018). Exploiting mit shocks in heterogeneous-agent economies: the impulse response as a numerical derivative. *Journal of Economic Dynamics and Control 89*, 68–92.

Carroll, C. D. (2006). The method of endogenous gridpoints for solving dynamic stochastic optimization problems. *Economics letters 91*(3), 312–320.

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems 2*, 303–314.

Den Haan, W. J. (2010a). Assessing the accuracy of the aggregate law of motion in models with heterogeneous agents. *Journal of Economic Dynamics and Control 34*(1), 79–99.

Den Haan, W. J. (2010b). Comparison of solutions to the incomplete markets model with aggregate uncertainty. *Journal of Economic Dynamics and Control 34*(1), 4–27.

Den Haan, W. J. and P. Rendahl (2010). Solving the incomplete markets model with aggregate uncertainty using explicit aggregation. *Journal of Economic Dynamics and Control 34*(1), 69–78.

Druedahl, J., R. Huleux, and J. Ropke (2026). Deep learning solutions of large non-convex lifecycle models. Technical report, University of Copenhagen.

Favilukis, J., S. C. Ludvigson, and S. Van Nieuwerburgh (2017). The macroeconomic effects of housing wealth, housing finance, and limited risk sharing in general equilibrium. *Journal of Political Economy 125*(1), 140–223.

Fella, G. (2014). A generalized endogenous grid method for non-smooth and non-concave problems. *Review of Economic Dynamics 17*(2), 329–344.

Fernández-Villaverde, J. (2026). Deep learning for solving economic models. *Journal of Economic Literature 64*(3), 829–875.

Fernández-Villaverde, J., S. Hurtado, and G. Nuno (2023). Financial frictions and the wealth distribution. *Econometrica 91*(3), 869–901.

Fernández-Villaverde, J., J. Marbet, G. Nuño, and O. Rachedi (2025). Inequality and the zero lower bound. *Journal of Econometrics 249*, 105819.

Fernández-Villaverde, J., G. Nuño, and J. Perla (2024). Taming the curse of dimensionality: quantitative economics with deep learning. Technical report, National Bureau of Economic Research.

Fernández-Villaverde, J., G. Nuno, G. Sorg-Langhans, and M. Vogler (2020). Solving high-dimensional dynamic programming problems using deep learning. *Unpublished working paper*.

Gomes, F. and A. Michaelides (2008). Asset pricing with limited risk sharing and heterogeneous agents. *The Review of Financial Studies 21*(1), 415–448.

Gopalakrishna, G., Z. Gu, and J. E. Payne (2026). Institutional asset pricing with segmentation and household heterogeneity. Working paper.

Gornemann, N., K. Kuester, and M. Nakajima (2021). Doves for the rich, hawks for the poor? distributional consequences of systematic monetary policy. Technical report, ECONtribute Discussion Paper.

Greenwood, J., Z. Hercowitz, and G. W. Huffman (1988). Investment, capacity utilization, and the real business cycle. *The American Economic Review 78*(3), 402–417.

Gu, Z., M. Laurière, S. Merkel, and J. E. Payne (2026). Global solutions to master equations for continuous-time heterogeneous-agent macroeconomic models. *Journal of Political Economy Macroeconomics*. Accepted, in press.

Han, J., Y. Yang, and W. E (2026). DeepHAM: A global solution method for heterogeneous agent models with aggregate shocks. *Quantitative Economics 17*(2), 297–341.

Hatcher, M. C. and E. M. Scheffel (2016). Solving the incomplete markets model in parallel using gpu computing and the krusell–smith algorithm. *Computational Economics 48*(4), 569–591.

Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks 4*(2), 251–257.

Huang, J. (2023). Breaking the curse of dimensionality in heterogeneous-agent models: A deep learning-based probabilistic approach. *SSRN Working Paper*.

Iskhakov, F., T. H. Jørgensen, J. Rust, and B. Schjerning (2017). The endogenous grid method for discrete-continuous dynamic choice models with (or without) taste shocks. *Quantitative Economics 8*(2), 317–365.

Jaimovich, N. and S. Rebelo (2009). Can news about the future drive the business cycle? *American Economic Review 99*(4), 1097–1118.

Kahou, M. E., J. Fernández-Villaverde, J. Perla, and A. Sood (2021). Exploiting symmetry in high-dimensional dynamic programming. Technical report, National Bureau of Economic Research.

Kase, H., L. Melosi, and M. Rottner (2025). Estimating nonlinear heterogeneous agent models with neural networks. BIS Working Papers 1241, Bank for International Settlements.

Khan, A. and J. K. Thomas (2008). Idiosyncratic shocks and the role of nonconvexities in plant and aggregate investment dynamics. *Econometrica 76*(2), 395–436.

Koby, Y. and C. K. Wolf (2020). Aggregation in heterogeneous-firm models: Theory and measurement. Working paper, Princeton University. July 15.

Krusell, P. and A. A. Smith (1997). Income and wealth heterogeneity, portfolio choice, and equilibrium asset returns. *Macroeconomic dynamics 1*(2), 387–422.

Krusell, P. and A. A. Smith, Jr (1998). Income and wealth heterogeneity in the macroeconomy. *Journal of political Economy 106*(5), 867–896.

Lee, H. (2026). A global dynamic nonlinear solution framework and the repeated transition method. Working Paper, University of Cambridge.

Maliar, L. and S. Maliar (2022). Deep learning classification: Modeling discrete labor choice. *Journal of Economic Dynamics and Control 135*, 104295.

Maliar, L., S. Maliar, and P. Winant (2021). Deep learning for solving dynamic economic models. *Journal of Monetary Economics 122*, 76–101.

Payne, J., A. Rebei, and Y. Yang (2025). Deep learning for search and matching models. Research Paper Series 25-05, Swiss Finance Institute.

Reiter, M. (2009). Solving heterogeneous-agent models by projection and perturbation. *Journal of Economic Dynamics and Control 33*(3), 649–665.

Senga, T. (2025, July). A new look at uncertainty shocks: Imperfect information and misallocation. Working paper.

Sunakawa, T. (2020). Applying the explicit aggregation algorithm to heterogeneous macro models. *Computational Economics 55*(3), 845–874.

Tauchen, G. (1986). Finite state markov-chain approximations to univariate and vector autoregressions. *Economics letters 20*(2), 177–181.

Terry, S. J. (2017). Alternative methods for solving heterogeneous firm models. *Journal of Money, Credit and Banking 49*(6), 1081–1111.

Valaitis, V. and A. T. Villa (2024). A machine learning projection method for macro-finance models. *Quantitative Economics 15*(1), 145–173.

Winberry, T. (2018). A method for solving and estimating heterogeneous agent macro models. *Quantitative Economics 9*(3), 1123–1151.

Winberry, T. (2021). Lumpy investment, business cycles, and stimulus policy. *American Economic Review 111*(1), 364–396.

Yang, Y., C. Wang, A. Schaab, and B. Moll (2025). Structural reinforcement learning for heterogeneous agent macroeconomics.

Young, E. R. (2010). Solving the incomplete markets model with aggregate uncertainty using the krusell–smith algorithm and non-stochastic simulations. *Journal of Economic Dynamics and Control 34*(1), 36–41.